《人工智能基础（A）》复习资料

前言：对《人工智能基础（A）》的 ppt 进行了梳理，对照历年卷考察的知识，整理了一份自认为“速通”的资料（不完全涵盖所有考点和知识点）。下面会对知识点的重要程度进行简单的评级（ABC），大概分为浏览有印象就行，要求掌握，重点。

一：初识人工智能（A）

人工智能的起源：约翰·麦卡锡等 4 位学者在 1956 年 8 月的达特茅斯学院夏季研讨会议提出了“人工智能”，人工智能诞生。

人工智能的两次冬天三次浪潮：第一次浪潮：1956 年达特茅斯会议，标志着人工智能的诞生；1970 年，受限于计算能力，人工智能进入第一个寒冬；第二次浪潮：1980 年，专家系统问世（可落地使用）；1990 人工智能计算机 DARPA 失败，政府投入缩减，进入第二次寒冬；第三次浪潮：2006 年，Hinton 提出深度学习的神经网络

人工智能的三大主义学派：进化学派 核心思想：借鉴生物进化的原理，如遗传、变异、自然选择等，让计算机程序或智能体通过模拟进化过程来优化自身的行为和性能。例如遗传算法，就是通过对种群中的个体进行选择、交叉、变异等操作，逐步搜索到最优解。

技术应用：在优化问题求解、机器人路径规划、神经网络结构设计等方面有广泛应用。比如在机器人路径规划中，利用进化算法可以让机器人在复杂环境中找到最优或较优的行走路径。

类推学派 核心思想：也可称为案例推理学派，主要基于相似性原理，即通过搜索与当前问题相似的过去案例，并将过去案例的解决方案适配到当前问题中。例如，当一个智能客服系统遇到一个新的客户咨询问题时，它会在已有的问题库中寻找相似问题及其答案来回复客户。

技术应用：在医疗诊断、法律案例分析、智能决策支持等领域应用较多。在医疗诊断中，医生可以借助过去类似病症的诊断经验和治疗方案来为新患者进行诊断和治疗。

贝叶斯学派 核心思想：以贝叶斯定理为基础，通过考虑先验概率和新的证据来更新对某个假设或事件的后验概率，从而进行推理和决策。例如在垃圾邮件过滤中，根据邮件中出现的词汇等特征，结合先验的垃圾邮件和正常邮件的概率分布，计算该邮件是垃圾邮件的后验概率。

技术应用：广泛应用于数据分类、预测、不确定性推理、机器学习中的模型评估和参数估计等。在机器学习的模型选择中，贝叶斯方法可以通过计算不同模型的后验概率来选择最优模型。

符号主义 符号表示-逻辑推理-搜索

认为：人类认知和思维的基本单元是符号，认知过程是以符号表示基础的一种运算，计算机是一个可用来模拟人的智能行为的物理符号系统。核心是构建符号表达知识。代表人物：艾伦·图灵、约翰·麦卡锡和马文·明斯基、艾伦•纽厄尔和赫伯特•西蒙等研究学者。主要成果有：机器定理程序、启发式算法、专家系统等

联结主义 基于神经网络-深度学习

联结主义是一种理解认知过程的理论框架。试图通过模拟大 脑的神经网络来解释人类的认知功能，如记忆、学习、语言处理和模式识别。目标是实现模拟人脑的结构，产生了神经网络方法。主要成果：神经网络深度学习，现代人工智能大语言模型

行为主义 基于环境交互强化学习
构建一个智能体，它关注智能体在环境中的行为，智能体以最大化奖励为目标，与环境不断交互来学习和适应，会根据行为的结果进行调整，主要方法包括（1）强化学习（2）Q-learning 和策略梯度方法 主要应用：游戏 AI、机器人控制、自动驾驶等。
现象理解：幸存者偏差（AI 的局限绝大多数起源于幸存者偏差现象）——沉默的大多数，就是忽略了筛选条件，把经过筛选的结果当成随机结果。（只看到幸存者却忽略了阵亡者，只搜集了一部分的信息，却遗漏了很多重要信息没看到，导致我们做判断的时候，会误判，会有偏差）

二：人工智能的数据基础（？）

内容繁杂且在历年卷中几乎没考察，建议学有余力的同学自己学习一下

三：人工智能的应用开发基础（A~B）

（没 python 基础的可以简单浏览一下对应章节的 ppt）
手写数字识别程序等代码的作用理解：

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 加载数据集
digits = load_digits()

# 将数据展平并分割为训练集和测试集
X = digits.images.reshape((len(digits.images), -1))
y = digits.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, ran-
dom_state=42)
```

```
# 标准化特征
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 训练K-近邻分类器
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

# 预测并计算准确率
predicted = knn.predict(X_test)
accuracy = accuracy_score(y_test, predicted)
print(f"Accuracy: {accuracy * 100:.2f}%")
```

函数定义：def 函数名（参数）：
导入模块的方法：import 模块名 ；from 模块名 import 功能名；from 模块名 import * ；import 模块名 as 别名；from 模块名 import 功能名 as 别名
安装库：方式一：pip install 库名 # 基础安装
方式二： conda install 库名 # Anaconda 环境
AI 算法库（具体看 ppt）：Pandas（数据处理与分析）
NumPy（数值计算与科学计算）
Matplotlib（可视化）
Scikit-learn（机器学习库）
深度学习框架：PyTorch；TensorFlow；PaddlePaddle；MindSpore

四：从问题求解到机器学习（A~B）

问题求解示例（A）:（1）装箱问题及求解——首次适应法（First-Fit）
（2）水壶倒水问题及求解——根据实际求解/贝祖定理/数论/最大公约数
算法方法学（A）：
（1）贪心法——主要用于求解最优问题 举例-找零钱问题
分阶段工作，在每一个阶段选择最好的方案，而不考虑其后的结果如何。
核心特性：可行性，局部最优，不可撤销性
缺点：不能确定最后解是最优的，也不能用于求解最大或最小问题。
优点：在算法的效率上，贪心法快速，程序实现需要的内存开销也较小。然而一旦被证明是正确的，其执行效率和速度有很大的优势。
（2）分治法——基本思想：分而治之 举例-对一组数据按大小快速排序
（3）回溯法（穷尽搜索法）——基本思想：从一条路往前走，能进则进，不能进则退回来，换一条路再试

通常使用递归实现　举例-N 皇后问题
（4）动态规划法　举例-斐波那契数列问题
基本思想：如果一个较大问题可以被分解为若干个子问题，且子问题具有重叠，可以将每个子问题的解存放到一个表中，通过查表解决问题
核心思想是以空间换时间
状态空间搜索问题求解（A）：　举例-八数码问题
定义：如果一个问题可以被定义为状态空间中的一个初始状态，通过一系列操作或动作转换直到到达目标状态，这样的问题就是状态空间（State Space）问题。
四要素：初始状态（Initial State）　目标状态（Goal State）
　　操作（Operators）　路径（Path）
启发式搜索问题求解（A）：　举例-走迷宫问题（A*搜索算法和贪心算法）
　　启发式搜索：是在搜索空间中查找问题解决方案的技术，它使用一个启发式函数来评价选择哪个分支可能会导向最佳解。
　　启发式函数可以看作是一种“直觉”或“猜想”，帮助算法决定下一步最有希望的路径。
其他搜索问题求解方法（A）：暴力搜索（穷举）；最大最小搜索算法（决策制定和博弈）；深度/广度优先搜索（一条路先走到底/历经每层，层层深入）；遗传算法（模仿自然选择）；模拟退火（寻找全局最优解）；约束满足问题求解（寻找满足所有约束的解）
机器学习及定义（A~B）：机器学习是人工智能的子集。机器学习已经成为一种重要的人工智能技术。从数据中发现规律并学习训练得到模型，并利用得到的模型进行推断和决策的方法
定义：已知 X 是输入空间，Y 是输出空间，Y 与 X 存在某种对应关系，Y 称为 X 的标签。　机器学习的目标是构建一个映射函数 f:X→Y，使得该函数在未知输入上能够尽可能准确地预测输出。
类型：有监督学习（所有的 X 都有 Y 对应）——分类和回归
　　无监督学习（没有 Y，学习 X 内部的知识）——聚类和降维
　　半监督学习（一部分 X 有 Y）强化学习（环境交互学习，最大化累积奖励）
数据预处理方法：

```python
# 导入内建数据集
from sklearn.datasets import load_iris

# 获取鸢尾花数据集
iris = load_iris()

# 获得 ndarray 格式的变量 X 和标签 y
X = iris.data
y = iris.target

# 获得数据维度
n_samples, n_features = iris.data.shape
print(n_samples, n_features)
```

```
150 4
```

数据标准化处理：

1）min-max标准化（对象名为MinMaxScaler），该方法使数据落到[0,1]区间：

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

2）Z-score标准化（对象名为StandardScaler），该方法使数据满足标准正态分布：

$$x' = \frac{x - \overline{X}}{S}$$

3）归一化（对象名为Normalizer，默认为L2归一化）：

$$x' = \frac{x}{\sqrt{\sum_j^m {x_j}^2}}$$

数据二值化：使用 Binarizer 对象实现数据的二值化（转化为布尔值）

```
# 二值化，阈值设置为3
from sklearn.preprocessing import Binarizer

results = Binarizer(threshold=3).fit_transform(X)

print("处理前：", X[1])
print("处理后：", results[1])
```

```
处理前： [4.9 3.  1.4 0.2]
处理后： [1. 0. 0. 0.]
```

标签编码和独热编码：

```
# 标签编码
from sklearn.preprocessing import LabelEncoder
LabelEncoder().fit_transform(['apple', 'pear', 'orange', 'banana'])
```

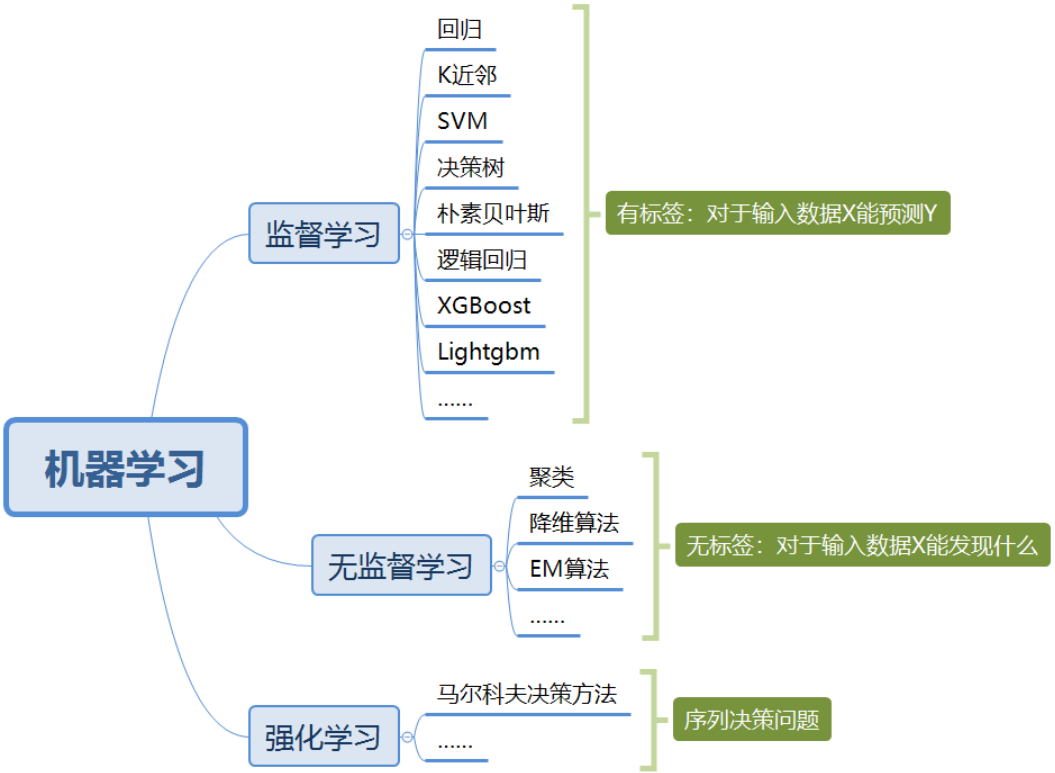
```
array([0, 3, 2, 1])
```

```
# 独热编码
from sklearn.preprocessing import OneHotEncoder

results = OneHotEncoder().fit_transform(y.reshape(-1,1)).toarray()

print("处理前：", y)
print("处理后：", results[1])
```

```
处理前： [0 0 0 
```



没有免费午餐定理（NFL）:Wolpert 和 Macerday1997 年在最优化理论中提出，指出“任何机器学习模型在所有问题上的性能都是相同的，其总误差和模型本身是没有关系的。一种算法（算法 A）在特定数据集上的表现优于另一种算法（算法 B）的同时，一定伴随着算法 A 在另外某一个特定的数据集上有着不如算法 B 的表现”——没有最优通解

监督学习方法：从数据中发现规律，并将这些规律应用于新的、未知的数据

类型：（1）回归问题——预测连续的数值，即输出标签是连续变量

（2）分类问题——要将输入数据划分到预定义的类别中，即标签是离散类别

典型流程：数据收集及预处理，选择算法，训练模型，模型评估和部署

模型评估：

混淆矩阵(Confusion matrix)是一种用于比较分类结果和实际值的可视化手段，常用来评估分类模型性能，可计算准确率、精确度、召回率、F1 分数、假阴性率及特异度等指标

实际类别	预测类别	
	1	0
1	TP (*True Positive*) 真正例	FN (*False Negative*) 假负例
0	FP (*False Positive*) 假正例	TN (*True Negative*) 真负例

- **精确率(Precision)：** $\frac{TP}{TP + FP}$
- **召回率(Recall)：** $\frac{TP}{TP + FN}$
- **F1分数(F1 score)：** $\frac{2TP}{2TP + FP + FN}$
- **特异度(Specificity)：** $\frac{TN}{TN + FP}$
- **假阴性率(False Negative Rate)：** $\frac{FN}{TP + FN}$

面临的挑战-过拟合：训练集上表现很好，预测时表现很差的现象
如何消除过拟合——提升泛化能力
方法：1.交叉验证 2. dropout：动态随机切断神经元的连接 3. 正则化：L1 范式（LASSO 回归）和 L2 范式（岭回归），优化损失函数 4. 不过度训练（实时监控）5. 特征选择（难度较大，往往事与愿违）6. 集成学习，即多种不同质模型的组合投票 7. 核心：数据质量与预处理技术（正道）
损失函数：用来计算对应的真实值与预测值之间差值的函数
常用损失函数：

（1） 回归损失函数，用于衡量回归系统的误差，常见的有如下几种：

均方误差（mean_squared_error）

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(T_i - Y_i)^2$$

平均绝对误差（mean_absolute_error）

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|T_i - Y_i|$$

平均绝对百分比误差（mean_absolute_percentage_error）

$$MAPE = \frac{1}{N}\sum_{i=1}^{N}|\frac{Y_i - T_i}{Y_i}| \times 100$$

均方根对数误差（mean_squared_logarithmic_error）

$$MSLE = \frac{1}{N}\sum_{i=1}^{N}[log(T_i + 1) - log(Y_i + 1)]^2$$

（2） 分类损失函数，用于衡量分类系统的误差

二进制交叉熵（binary_crossentropy）：用于二分类问题，对应的激活函数是sigmoid

$$Loss = -\frac{1}{N}\sum_{i=1}^{N}[Y_i \times logT_i + (1 - Y_i) \times (1 - T_i)]$$

多分类交叉熵（categorical_crossentropy）：用于多分类问题，对应的激活函数是softmax

$$Loss = -\frac{1}{N}\sum_{i=1}^{N}Y_i \times logT_i$$

回归模型：一种统计学模型，用于研究变量之间的定量关系，特别是一个或多个输入自变量 （也称为特征或预测变量）对输出因变量（也称为目标变量或响应变量）的影响

线性回归模型：使用最小二乘法（OLS）估计模型参数（β 值），目标是最小化残差平方和（RSS）；通过统计指标（如均方误差 MSE、R^2 ）评估模型性能

逻辑回归（Logistic regression）是一种广义线性回归——用于二分类——设定一个阈值（如 0.5），当因变量大于阈值时，为 1 否则为 0，完成分类

分类模型类型：二分类模型——逻辑回归（sigmoid 函数）；支持向量机

多分类模型——决策树；随机森林；多类 SVM

强化学习： （运用 Q-Learning 算法等）

马尔可夫决策过程(MDP) 由五个主要部分组成，以元组(S， A， P， R， γ)

表示

- S——状态集，当前状态
- A——行动集合
- P——状态转移概率函数
- R——奖励函数，可正可负
- γ——折扣因子函数，平衡短期和长期奖励

六：数据的聚类和降维技术（A~B）

无监督学习:机器学习的一种方法，旨在从没有标签的数据中识别潜在的模式或结构

无监督学习技术:聚类——目标是将相似的数据点分组在一起，形成"簇"（cluster）。

降维——降维将高维数据转换为低维表示，同时巧妙地保留了原始数据中最重要的信息。

异常检测——发现潜在的异常样本，通常指示错误、欺诈或其他重要事件。

聚类：核心思想是通过计算数据点之间的相似度或距离（欧氏距离），将相似的数据点聚集在一起形成簇——K-means 算法　对照：KNN（K-Nearest Neighbors）——监督学习

降维：将高维数据转换为低维表示的过程，以简化数据集并降低计算复杂性

主成分分析（Principal Component Analysis，PCA）是最常用的降维技术之一

主要步骤为：(1)数据中心化；(2)计算协方差矩阵；(3)计算特征向量和特征值；(4)选择主成分（保留最大方差的方向，根据累积方差的贡献率选 k 个主成分）

无监督学习案例——鸢尾花数据（理解代码）

七：深度网络基础组件（B~C）

神经元的“全或无”现象理解：只有当外来刺激有足够大的强度，才能引起神经元细胞的兴奋并产生动作电位，但再增加刺激强度并不会导致动作电位的幅度发生变化。

深度学习：机器学习的一个子集

基本思想：通过构建多层网络，对目标进行多层表示，以期通过多层的高维特征来表示数据的抽象语义信息，从而获得更好的特征鲁棒性

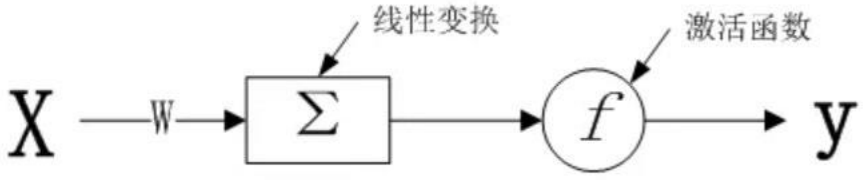
多层网络结构：输入层、隐藏层和输出层

常用深度学习模型：多层感知机（MLP）、卷积神经网络（CNN）、循环神经网络、Transformer

三个重要概念：损失函数（比较）、反向传播（反馈）、梯度下降（调整后下一轮）。

感知机模型（必须会计算）：它可以被视为一种最简单形式的前馈式人工神经网络，是一种二元线性分类器（下图 b 为偏置）

输入X可以是一个任意维度n的向量 $X=[x_i,x_2,...,x_n]$，W是X的权重向量，$W=[w_1,w_2...,w_n]$,网络 $\sum=WX^T+b$ 完成线性变换。 f 是激活函数，完成非线性变换。



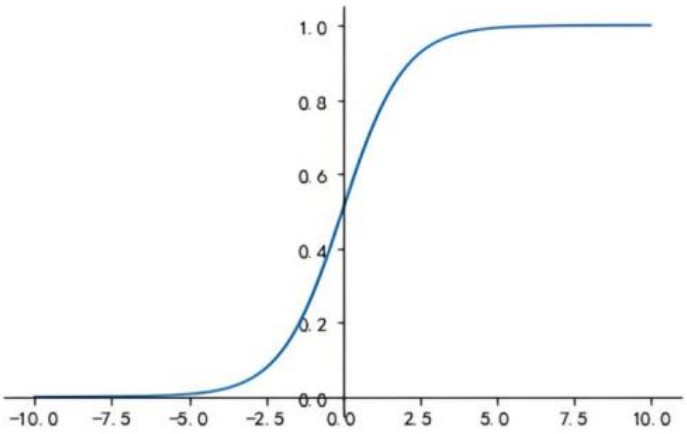
感知机的局限：无法用一条直线实现分类——无法解决“异或问题”

激活函数 f ——解决感知机局限的方法

作用：通过非线性变换，使神经网络具备非线性特性，以解决一些非线性任务。

Sigmoid

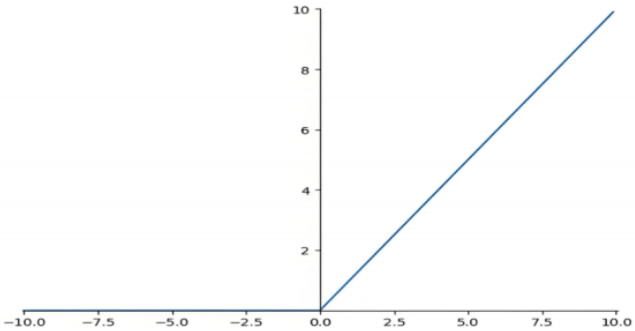
$$f(x)=\frac{1}{1+e^{-x}}$$



Softmax：sum=$\sum_j e^{x_j}$　$S_j=e^{x_j}$/sum

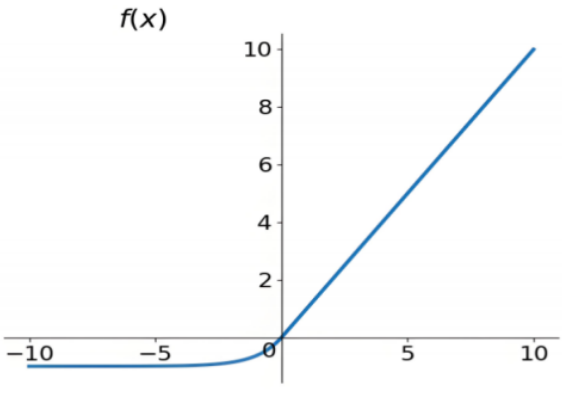
ReLU（目前隐含层中最为常用）

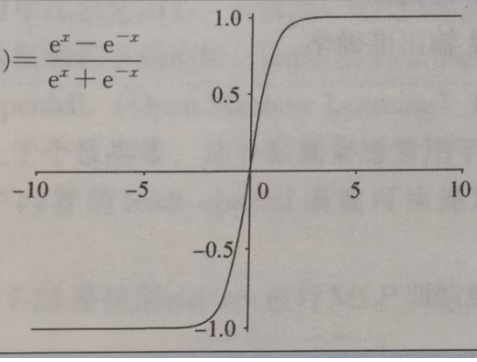
$$f(x)=max(0,x)$$



ELU

$$f_i(a_i)\begin{cases} x & \text{if } x>0 \\ \alpha(e^x-1) & \text{if } x\le 0 \end{cases}$$



(2) Tanh	特性
$f(x)=\frac{e^x-e^{-x}}{e^x+e^{-x}}$ 	1. 非线性 2. 在－3与3之间，优化比较明显 3. 在－3与3之外，优化不明显 4. 值域在－1~1之间，对称算法，解决了Sigmoid的局限 5. 梯度比Sigmoid陡峭，易形成梯度崩塌

多层感知器 MLP（单隐含层）:至少包含三层：一个输入层，一个隐藏层和一个输出层

只有一个隐含层的叫浅层学习网络，隐含层大于一层的叫深度学习网络

反向传播算法（BP 算法）：一种常见的人工神经网络学习算法，特别适用于多层前馈神经网络的训练。该算法由学习过程中的信号正向传播与误差的反向传播两个过程组成

特点：（1）自适应、自主学习；（2）较强的非线性映射能力；（3）严谨的推导过程；（4）较强的泛化能力

局限性：（1）易陷入局部最小值；（2）收敛速度慢；（3）隐节点选取缺乏理论指导；（4）学习新样本时可能遗忘旧样本

梯度下降法：以最快的速度找到函数局部最小值的优化算法

最小值寻找过程：（1）方向的选择-梯度 （2）前进的步长-学习率 （3）何时终止 - 梯度≈0

如何应用：设计损失函数，沿着损失函数梯度的反方向更新网络参数

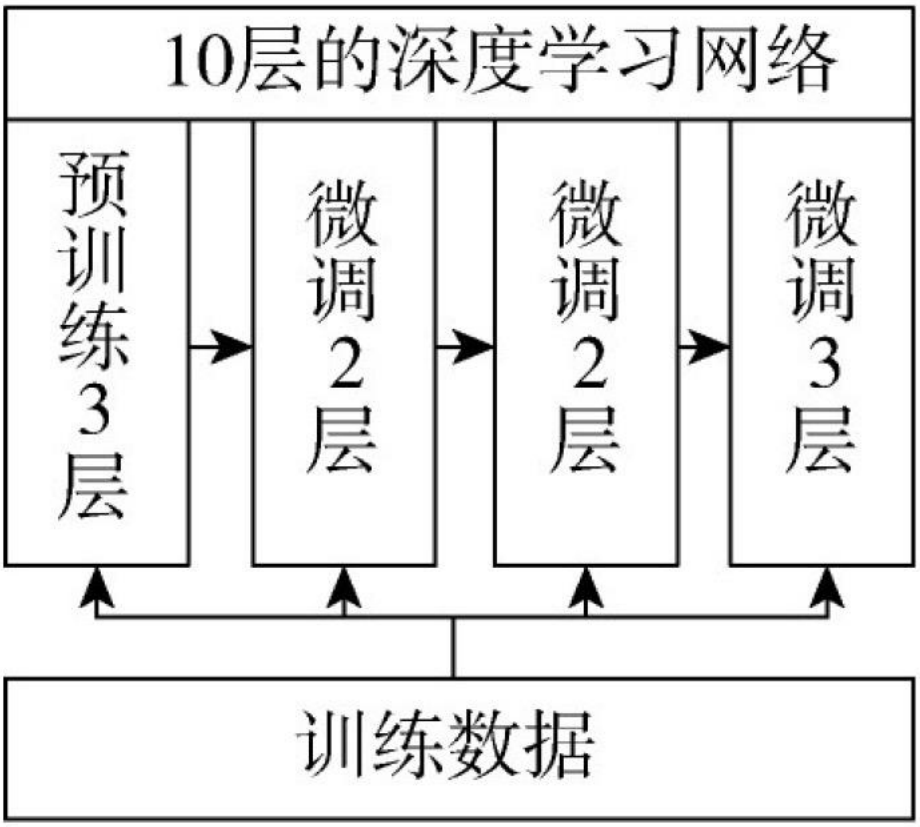
多隐含层 MLP：具体见第八章

深度学习的三个要素: 算法+数据+算力

目标：从海量的数据中自动寻找特征

算法：误差反向传播算法（BP 算法）网络拓扑结构：带计算的网络

激活函数：f（x）=... 优化器：梯度下降法 训练方式：预训练—微调

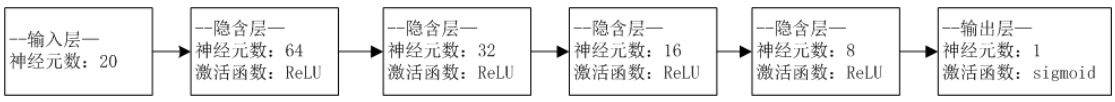


图像领域的数据集：ImageNet、CIFAR-10、Labelme、Youtube-8M、CelebFaces

算力：在广义上，算力是计算机设备或计算/数据中心处理信息的能力。狭义上，算力是指一台计算设备理论上最大的每秒浮点运算次数。

八：卷积神经网络（C）

多隐含层 MLP 的实现：案例 1：用 TensorFlow 实现 MLP



1. 导入必要的库

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np
```

2. 准备数据

```
np.random.seed(0)
# 1000个样本，每个样本20个特征
x_train = np.random.random((1000, 20))
# 1000个标签，0或1
y_train = np.random.randint(2, size=(1000, 1))
```

3.定义模型结构

```
model = Sequential([
# 输入层-隐含层1，20个输入特征，64个神经元，ReLU激活函数
Dense(64, activation='relu', input_shape=(20, )),
# 隐藏层2，32个神经元，ReLU激活函数
Dense(32, activation='relu'),
# 隐藏层3，16个神经元，ReLU激活函数
Dense(16, activation='relu'),
# 隐藏层4，8个神经元，ReLU激活函数
Dense(8, activation='relu'),
# 输出层，1个神经元，sigmoid激活函数用于二分类
Dense(1, activation='sigmoid')
])
```

4. 编译模型

```
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

在训练模型之前，需要调用model.compile 编译模型，指定优化器adam、损失函数binary_crossentropy'和评估指标accuracy。

5. 训练模型

```
model.fit(x_train, y_train, epochs=10, batch_size=32)
```

调用model.fit 训练模型时，指定输入x_train，标签y_train，迭代次数epochs=10，每批投喂数据条数batch_size=32。

6. 评估模型

```
score, acc = model.evaluate(x_test, y_test, verbose=0)
print('Test loss: ', score)
print('Test accuracy: ', acc)
```

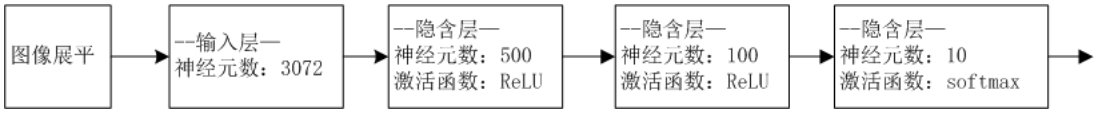
- 模型损失：loss
- 准确率：acc

7. 使用模型进行预测

```
prediction = model.predict(x_new)
print(prediction)
```

假设x_new是一个新的数据样本，调用model.predict进行预测。

案例 2：PyTorch 实现 CIFAR-10 的识别分类（具体代码见 ppt）



1. 导入必要的库

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

2. 数据加载和预处理

```
# 数据预处理
transform = transforms.Compose([
  # 输出张量格式的数据
  transforms.ToTensor(),
  # 归一化RGB三个通道的数据，平均值=0.5，标准差=0.5
  transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

可以将多个数据变换组合在一起，用一个列表结构[...]来列出所有的转换。

14

```
# 加载数据集
train_dataset = datasets.CIFAR10(root='./data', train=True,
download=True,transform=transform)
test_dataset = datasets.CIFAR10(root='./data', train=False,
download=True,transform=transform)
```

datasets.CIFAR10：装载 CIFAR10 数据集，如果本地没有该数据集，则会自动下载。

```
# 批量加载数据用于后续训练加速。并打乱数据次序
train_loader = DataLoader(train_dataset, batch_size=64,
shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64,
shuffle=False)
```

DataLoader：PyTorch 内置数据加载器，训练时批量投喂数据。

3. 定义 MLP 模型

```
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        # 输入层节点数 = 32*32*3（图片尺寸）
        self.fc1 = nn.Linear(32 * 32 * 3, 500)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(500, 100)
        self.fc3 = nn.Linear(100, 10) # 输出层节点数 = 10（类别数）
     def forward(self, x):
        x = x.view(-1, 32 * 32 * 3) # 展平图片，降维到1维
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x
model = MLP()
```

4. 参数设置

```
# 设定超参数
learning_rate = 0.001 #学习率
epochs = 10 #迭代次数
# 损失函数和优化器
criterion = nn.CrossEntropyLoss() #自带softmax
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
```

损失函数采用交叉熵损失算法

5. 训练模型

```
# 训练模型
for epoch in range(epochs):
    for images, labels in train_loader:
        optimizer.zero_grad() #很重要，误差不累积，置零
        outputs = model(images) #前向计算
        loss = criterion(outputs, labels) #计算损失
        loss.backward() #误差反向传播
        optimizer.step() #用选定的优化器更新模型参数

    print(f'Epoch [{epoch + 1}/{epochs}], Loss: {loss.item():.4f}')
```

每迭代一次，都显示一次训练的结果

展示训练的过程——随着训练次数增加，损失函数逐渐减小。

```
Epoch [1/10], Loss: 1.1324
Epoch [2/10], Loss: 1.1708
Epoch [3/10], Loss: 1.5662
Epoch [4/10], Loss: 1.1874
Epoch [5/10], Loss: 1.0902
Epoch [6/10], Loss: 0.8871
Epoch [7/10], Loss: 1.5167
Epoch [8/10], Loss: 1.3621
Epoch [9/10], Loss: 1.0083
Epoch [10/10], Loss: 0.5777
```

6. 评估模型

```
model.eval() # 将MLP切换到评估模式
with torch.no_grad():
    correct = 0
    total = 0
    for images, labels in test_loader:
        outputs = model(images)  # 前向计算输出结果（预测的分类概率）
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
    print(f'Accuracy of the network on the 10000 test images: {100*correct/total}%')
```



模型输出

```
Accuracy of the network on the 10000 test images: 53.04 %
```

卷积：

连续函数的卷积（自变量相加为 x）：

$$h(x) = \int_{-\infty}^{+\infty} f(\tau)g(x-\tau)d\tau$$

离散函数的卷积（自变量相加为 n）：

$$h(n) = (f \cdot g)(n) = \sum_{m=-\infty}^{+\infty} f(m)g(n-m)$$

感受野：物体在视网膜处投射出影像后，在视觉信号向大脑传导的过程中，并不采用所有神经元的全连接的方式，而是首先进行局部处理，经过多个层次的局部处理提取出特征值，再逐层传递，这个局部的大小范围就是“感受野”

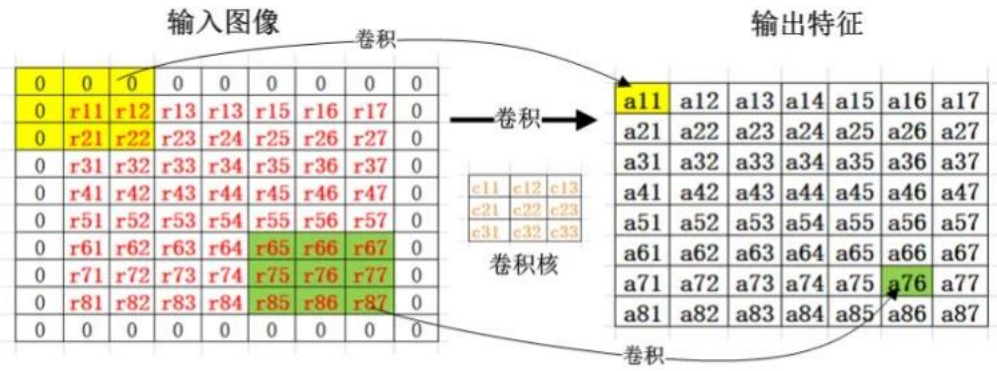
卷积神经网络：基于卷积算法思想和感受野概念构成的神经网络

数字化图像：将模拟图像转化为计算机可处理数字形式的图像

彩色图像表示：数字图像每个像素用 RGB 三原色表示，每种颜色强度用 1 字节储存，取值 0-255，每个像素可有 16777216 种颜色，能满足计算机区分颜色需求

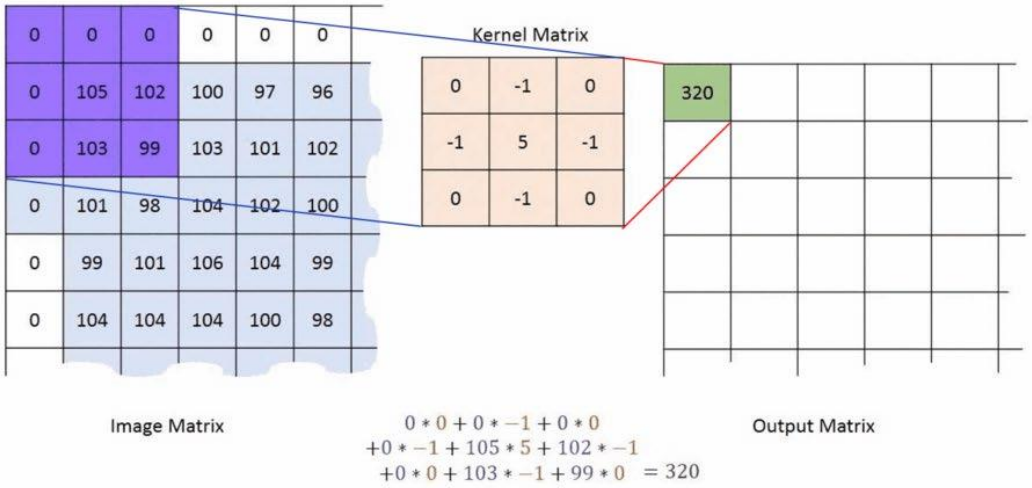
灰度图像表示：灰度图像用 1 个亮度通道（1 字节）表示，取值 0－255，0 为最黑，255 为最亮。黑白图像是特殊灰度图像，也用 1 字节表示，黑 0 白 255。早期因计算机内存小，黑白图像常用 1 位表示，黑 0 白 1

卷积运算：卷积运算就是通过设计一系列大小适中的卷积核（感受野），对数字图像的各个通道分量（二维矩阵）进行卷积，提取特征值的过程。常用的卷积核大小为 3×3、5×5、7×7



卷积运算

补齐操作：补 0



卷积过程中的其他操作：计算其他颜色通道；采用新的卷积核；生成 n 组特征向量

卷积运算运算中的三个主要参数：（1）卷积核形状；（2）步长：决定运算的速度；（3）卷积核数量:决定每一层能提取的特征数

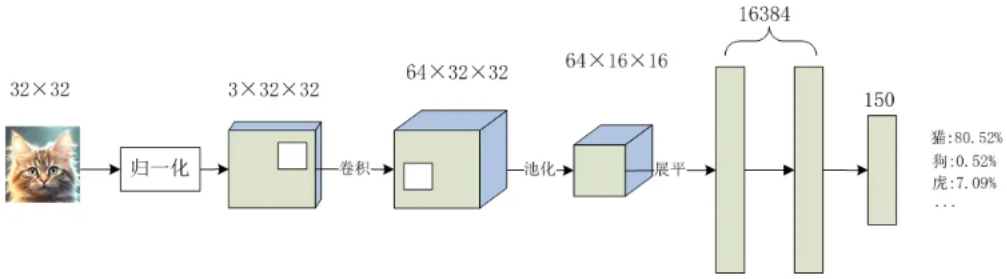
池化（pooling）：池化也称下采样，其作用就是缩小特征图的尺寸，减少计算量。池化的原理是用某一图像区域子块的统计信息包含该子块全局信息。

池化操作主要有最大池化、平均池化、随机池化、L2 范数池化、K-max 池化和全局平均池化等。CNN 常用 2×2 区域进行池化

归一化：即将特征值的取值范围压缩在 0~1 之间。

最简单的卷积神经网络-CNN：三要件：卷积、池化、展平

一个最典型的卷积神经网络是由卷积层（多次卷积运算、池化运算和归一化）、全连接层和输出层组成的前馈神经网络。



较复杂的卷积神经网络-AlexNet 拓扑结构

CNN 模型：本质——层次特征学习

层次特征学习意义：1、抽象特征具有很强的不变性，比如一般的人特征不会随外貌动作的改变而改变，还是一个人；2、从数据中自动获得；3、简化设计，避免人为设计的主观性

展平：用于将多维的数据降为一维，构建 MLP，完成最后的分类任务。

独热码（One-Hot code）：在分类问题中，我们不能将不同的分类用 1,2,3…这种有序的数字来表示，只能通过独热码这样的向量矩阵来进行编码，供计算机识别计算，并用所在索引位置的概率值来预测最终的分类结果。

代码案例：见 ppt

用途：图像分类；物体检测；图像分割；回归问题

核心：通过卷积核的卷积运算把相邻的数据进行特征分析，而忽略较远的数据，只要目标任务符合这样的特征均可采用 CNN 进行建模

九：循环神经网络 RNN（C）

三种神经网络：（1）前馈神经网络——多层感知机 MLP 卷积神经网络 CNN

（2）反馈神经网络——循环神经网络 RNN 长短程记忆网络 LSTM Hopfield 网络 波尔兹曼机

（3）图网络——知识图谱 社交网络 城市交通

MLP 对序列数据建模存在的问题：1、网络结构很庞大 2、输入层大小不确定 3、输出层很庞大 4、缺少语义的前后关系 5、多层感知机更擅长分类问题

循环神经网络（Recurrent Neural Network，RNN）：是一种特殊类型的反馈神经网络，专门用于处理序列数据。

RNN 的核心思想在于其循环结构，使得网络能够捕捉和利用序列中的顺序依赖性信息。

问题：无法学习太长的序列，“很快忘记前面说过的话”

RNN 的基本单元是一个具有循环连接的神经网络层。这个循环连接允许网络在处理每个时间步的数据时，能够利用之前时间步的信息。具体来说，RNN 在每个时间步都会接收一个输入，并产生一个输出，同时其内部状态（隐藏状态）会被更新并传递到下一个时间步

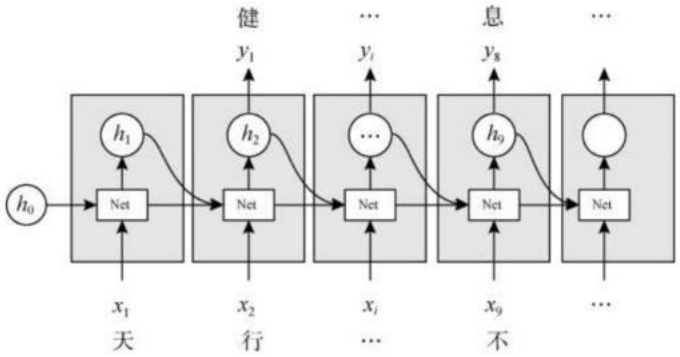


图 9-3　RNN按序列展开实例

RNN 分类：（1）1-1——图像去噪 数据加密（2）1-多——关键词　写作拼写纠正（3）多-1——语句情感分类 垃圾邮件识别 股票张跌判断 关键词提取（4）同步多对多——命名实体识别 词性标注（5）异步多对多——语句预测 机器翻译 文本摘要 聊天机器人 语音识别 股票走势预测

例10-1：已知拓扑结构为同步多对多RNN（基本结构见图 RNN基本逻辑结构），输入层、隐含层（一层）、输出层的神经元均为一个，激活函数均为 ReLU，$W=[0.5,0.1,0.2]$，$H=[1]$，$V=[3]$，$S_0=0$，$\alpha=0$，$\beta=0$，对 X =[[1,1,1]，[2,2,2]，[3,3,3]] 的输入序列，计算其输出序列 Y 。

解： $X_1=[1,1,1]$　$X_2=[2,2,2]$　$X_3=[3,3,3]$

$$S_1=f(W\bullet X_1^T+H\bullet S_0)=f(0.8)=0.8$$

$$Y_1=h(V\bullet S_1)=h(2.4)=2.4$$

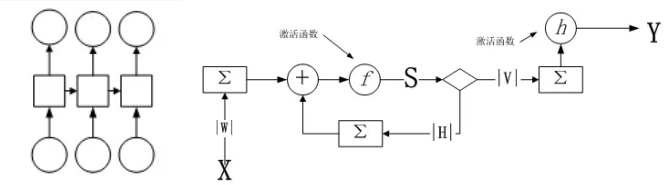
$$S_2=f(W\bullet X_2^T+H\bullet S_1)=f(1.6+0.8)=2.4$$

$$Y_2=h(V\bullet S_2)=h(7.2)=7.2$$

$$S_3=f(W\bullet X_3^T+H\bullet S_2)=f(2.4+2.4)=4.8$$

$$Y_3=h(V\bullet S_3)=h(7.2)=14.4$$

$$\therefore Y=[2.4,7.2,14.4]$$



$$Y_t=h(V\bullet S_t+\alpha)$$

$$S_t=f(W\bullet X_t+H\bullet S_{t-1}+\beta)$$

长短期记忆网络（Long Short-Term Memory， LSTM ）：专门设计用于解决长期依赖的问题

由输入门、遗忘门、输出门及记忆状态组成

局限 ：1、计算复杂 2、顺序处理，无法进行并行化计算 3、计算慢，实时响应难

十：人工智能应用开发方法（A）

大部分内容与前面重复，可简单浏览下 ppt

十一：自然语言处理 NLP 建模（B~C）

图灵测试：多名评委在隔开的情况下，通过设备向一个机器人和一名人类随意提

问.多次问答后，若超过 30%的人不能确定被测者是人还是机器，那么，该机具备人类智能。

分词 Token：令牌，可以是一个字，也可以是一个词，或者是一个字母，甚至是一个字节，要看具体的情况。

本质上，一个“Token”就是通过分词技术（工具）将一句话分割成的最小单位，是一个特定的自然语言处理模型能处理的最基本元素

eg.按字分“黄-山-落-叶-松-叶-落-山-黄” 词汇表 5 个 token，句子 9 个 token

词向量和词嵌入:词向量就是词的特征分布，是 NLP 模型层与层之间进行信息传递的数据形式。（二者为同一个东西，区别在于词向量是指数字编码技术，词嵌入是指 NLP 网络之间的数据存在形式）

句子的独热码表示：方式二：一维的稠密矩阵 [2,2,2,2,1]

黄	[1,0,0,0,0]
山	[0,1,0,0,0]
落	[0,0,1,0,0]
叶	[0,0,0,1,0]
松	[0,0,0,0,1]
叶	[0,0,0,1,0]
落	[0,0,1,0,0]
山	[0,1,0,0,0]
黄	[1,0,0,0,0]

方式二：二维的稀疏矩阵

文本相似度：一般地，语义相近的词在向量空间上具有相近的位置

（1）余弦相似度

设两个文本的向量为 A 和 B ，那么 A、B 的余弦相似度计算公式如下：

$$\text{Cosine Similarity}=\cos\theta=\frac{A\cdot B}{\|A\|\,\|B\|}$$

$A\cdot B$ 表示向量 A 和 B 的点积，$\sum_{i,j} A_i\cdot B_j$（i =1~ m ， j =1~ n ）

而||A|| ||B||表示A的模和B的模乘积，$\sqrt{\sum_{i=1}^{m}(A_i)^2}\times\sqrt{\sum_{j=1}^{n}(B_j)^2}$

（2）曼哈顿距离：$d=\sqrt{\sum_{i=1}^{n}|x_i-y_i|}$ （距离越短，则两个文本越相似）

（3）欧式距离：$d=\sqrt{\sum_{i=1}^{n}(x_i-y_i)^2}$ （距离越短，则两个文本越相似）

（4）Jaccard 相似度：$\mathrm{EJ}(A,B)=\frac{A\cdot B}{\|A\|^2+\|B\|^2-A\cdot B}$

其中，$A\cdot B$ 表示向量乘积，||A||和||B||分别表示向量 A 和 B 的模。

狭义定义：$J(A,B)=\frac{|A\cap B|}{|A\cup B|}$（值越大，则两个文本越相似）

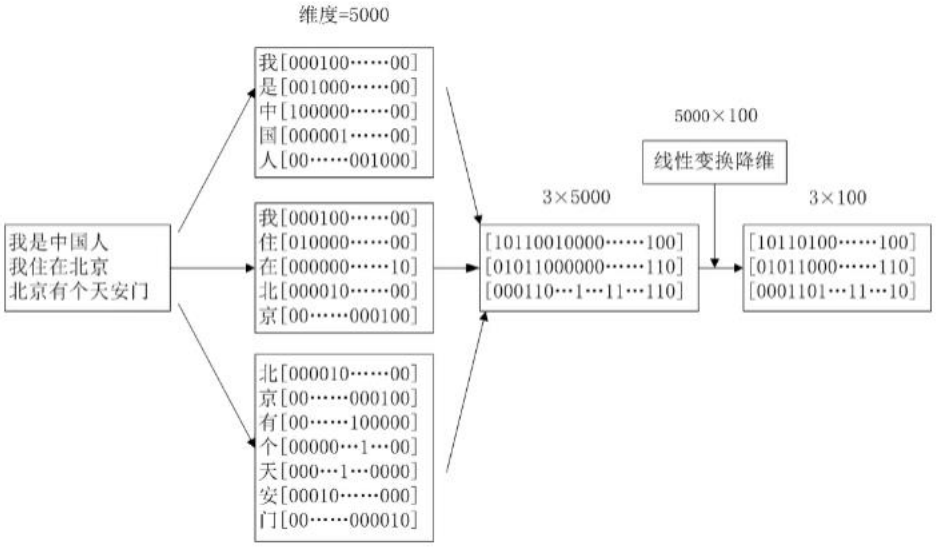
经典 NLP 模型：

（1）词袋模型（Bag of Words，简称 BoW）：自然语言处理和信息检索中的一种常用文本表示方法。

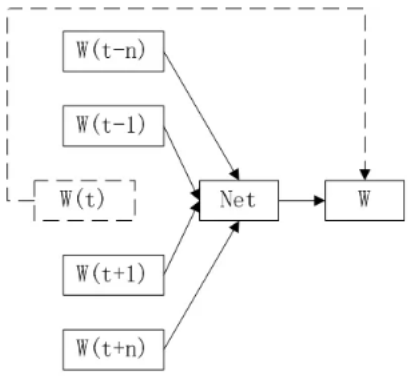
它将文本表示为一个词的集合，忽略词语的上下文关系，只计算词语的出现频率和其他统计值。

（2）Word2Vec：将词汇表中的每个词用固定长度的向量来表示，维度是最主要的一个参数。

如果词汇表的大小是 N，那么每个词向量的维度就是 N。如果一个句子长度是 M，那么这个句子可以表示为 M*N 的向量矩阵。（下图词汇表为 5000，有三个句子）

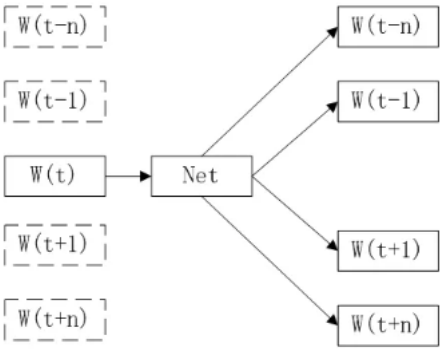


主要实现方式：1.CBoW（Continuous Bag of Words），使用给定上下文词（周边词）来学习预测中心词



（W（t）为中心词,n=窗宽，Net 是要训练的网络）

2.Skip-Gram：通过给定的中心词，来学习预测上下文词



（Word2Vec 在推理时不需要使用 Net，只需词向量）

（3）Seq2Seq：其核心思想是使用一个编码器网络将输入序列（如源语言句子）编码为一个固定维度的向量或一系列隐状态，然后使用一个解码器网络从这个向量或隐状态出发，逐词生成目标序列 （如目标语言句子）。整个过程无须人工设计复杂的语言规则，而是让神经网络自行学习如何进行有效的序列转换



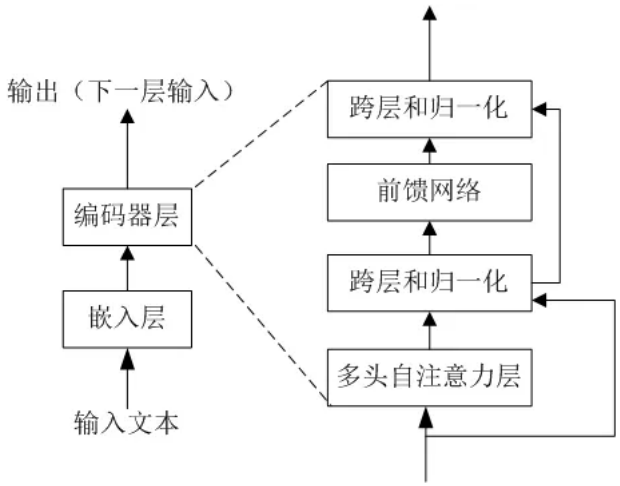
（编码器和解码器都是深度学习网络）

Transformer：是一种基于注意力机制的序列模型，与 CNN,RNN 不同，仅使用自注意力机制（self-attention）来处理输入序列和输出序列，可以并行计算，极大地提高了计算效率

技术特点：（1）序列到序列的编码器-解码器，自回归语言生成技术

（2）注意力机制

编码器　单个编码器层的展开



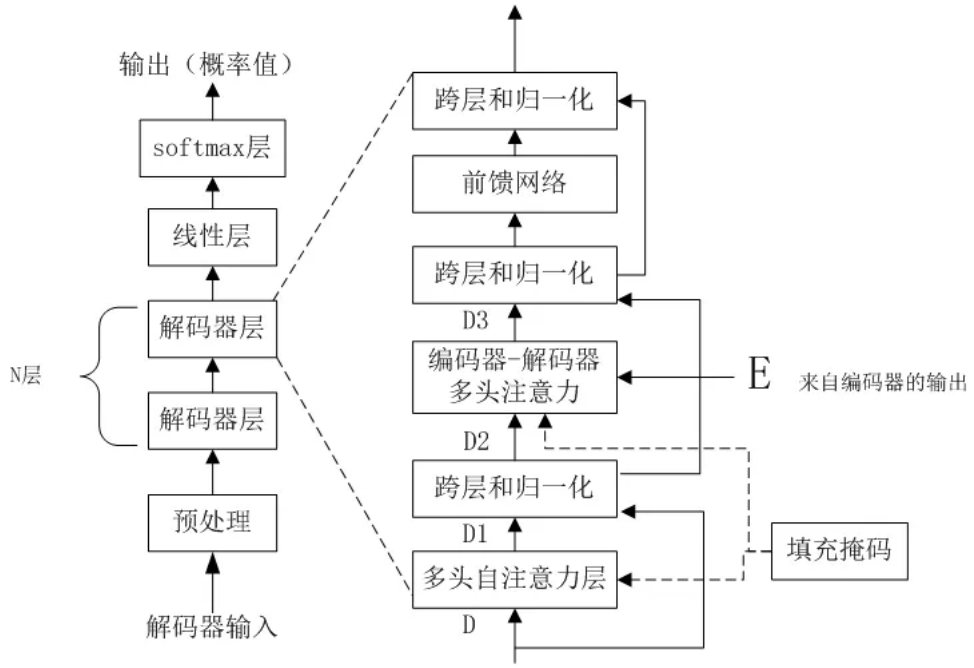
嵌入层内部实现原理：（1）分词：S=[我，在，学，人，工，智，能]

（2）增加位置序列 P=[1,2,3,4,5,6,7]

（3）编码：在嵌入层网络，完成对 S 和 P 的编码工作，然后将 S 与 P 进行拼接，输出向量 X

逐位置前馈网络：是一个两层的前馈神经网络，每个位置的词向量都独立地经过这个网络进行变换，而不是通常的将多维输入向量平铺展开成一维的向量。在上图中展示的是采用两个一维卷积网络（kernel_size=1）

解码器　单个解码器层的展开



关于解码器的输入：（1）编码器的输入+解码器的输出 比如：白日依山尽　黄河入海流　输入=白日依山尽 输出=黄河入海流　编码器的输入=白日依山尽黄河入海流

（2）填充掩码　输出采用自回归算法，为防止看到未来数据，所以需要增加一个掩码操作，把当前位置之后的数据掩盖掉，解码器边生成边向右移动，直到出现终止条件

多头：自注意力机制

在注意力机制中，***Q***（Query，查询）、***K***（Key，键）和***V***（Value，值）是三个关键概念：

Q：指当前需要关注或查询的信息点，用于与***K***进行相似度计算，以确定哪些键（及其对应的值）是重要的，并据此分配注意力权重。

K：指输入数据中的各个部分或特征，用于与***Q***进行匹配，以确定哪些部分与当前查询最为相关，***K***的得分将用于计算注意力权重，进而决定从***V***中提取多少信息。

V：指实际被加权求和以生成输出的参数，它包含了输入数据中的具体信息。***V***根据注意力权重进行加权求和，以生成最终的注意力输出。

$Q=XW^Q$

$K=XW^K$

$V=XW^V$

（1）线性变换

（2）计算注意力分数 $score=QK^T$　score 是表示 Q 和 K 之间的相似度

（3）归一化注意力分数 $\text{attention_weigh}=\text{softmax}\left(\frac{score}{\sqrt{D}}\right)$　D=512

（4）最终注意力分数 $Z=\text{attention_weigh}\cdot V$（加权和点积）

多头注意力即将 Q、K、V 分为 N 个头，每个头使用不同的权重参数进行自注意力计算，所有这些头的注意力分数会最终拼接起来，产生最终的多头注意力分数。

N 通常设为 8
多头注意力机制是在自注意力机制的基础上扩展的 （1）将输入在多个子空间里捕获多种特征，每一个头关注某一个特征，如词性、成分、词义、位置、关键词（2）可进行并行处理，加快训练。
三种类型的 transformer 模型：
（1）BERT——仅采用编码器结构而无解码器
强项：语义的理解能力，通过注意力机制，可同时考虑单词的上下文信息多任务训练方式
弱项：自然语言生成能力。因此需要搭配其他语言模型或进行微调
应用：阅读理解、完形填空、命名实体识别、QA、机器翻译、情感分类等
（2）GPT——仅采用解码器结构而无编码器
强项：自然语言生成能力，通过自回归技术生成对话
弱项：语义的理解能力，因此需要搭配其他语言模型或进行微调
应用：内容生成、问答系统、机器翻译、代码补全等
（3）T5（Text-To-Text Transfer Transformer）——同时采用解码器和编码器
强项：将 NLP 任务统一到一个框架（序列到序列）下进行训练，能力比较平衡
弱项：参数规模小，模型结构复杂，计算资源需求高，特定任务表现差
应用：翻译、摘要、问答等

十二：大语言模型与生成式人工智能（A~B）

本章节主要是概念辨析与记忆
AIGC：Artificial Intelligence Generated Content，人工智能生成内容
缺乏原创性和可解释性
LLM：Large Language Model，大语言模型（专门用于执行 NLP 任务的语言模型）
大的含义：训练数据大；参数规模大；耗资巨大：
涌现能力：当一种系统在复杂性增加到某一临界点时，会出现其子系统或较小规模版本中未曾存在的行为或特性
GAI：Generative Artificial Intelligence，生成式人工智能
特指能生成全新内容的 AI，其生成的内容就是前面所提的 AIGC， AIGC 侧重内容生成的来源，而 GAI 侧重 AI 系统的功能特点
AGI：Artificial General Intelligence，通用人工智能
训练要求：1、领域无关 2、任务无关
GPT：Generative Pre-trained Transformer，生成式预训练变换器，一种基于 Transformer 结构的预训练模型
ChatGPT：Chat Generative Pre-trained Transformer，OpenAI 推出的采用 GPT 架构的聊天机器人
预训练（海量，无监督，目标是通用能力）：不针对特定任务，而是用大规模的数据训练模型，使其具备各方面的基础能力，类似于我们人类的通识教育
大语言模型的预训练是通过在大规模未标注文本数据上，利用无监督学习（如预测下一个词或掩码词）来捕捉文本中的规律，最终生成一个能够理解并生成语言的通用模型。
微调（小量，有监督，目标是任务适配）：利用预训练模型已经学到的通用特征表示，针对特定任务微调使模型能够更好地适应某个任务。
微调的方式 1、对预训练的所有模型参数进行调整 2、保持预训练模型的参数不变，而增加一个小型网络 3、冻结预训练模型的大部分参数，只训练其中的

少部分参数

AI 幻觉（AI Hallucination）是指人工智能系统（尤其是大语言模型或生成式 AI）在生成内容时，输出与事实不符、逻辑混乱或完全虚构的信息

为什么会出现 AI 幻觉？ 1.训练数据局限：模型基于概率生成文本，若训练数据不足或存在偏差，可能编造答案。 2.过度泛化：AI 倾向于提供“看似合理”的回应，即使缺乏真实依据。 3.提示词（Prompt）引导：模糊或矛盾的指令可能触发虚构内容（例如要求 AI 描述未发生的事件）

十三：预训练微调与多模态模型（A）

私人助手定制（有余力看 ppt 浏览）

微调方式分类：

（1）全微调 （Full Fine-Tuning）

基础模型的所有参数都参与微调，适用于有全新的足够大的数据。微调时需要对原生模型的知识进行重构以适应新的环境，比如从英文语境转到中文语境。

（2）部分微调 （Partial Fine-Tuning）

冻结基础模型部分层的参数，调整非冻结参数。

（3）高效参数微调 （Parameter-Efficient Fine-Tuning，PEFT）

这是目前最常用的微调方式，它通过微调少量参数来达到接近全量微调的效果。常见的 PEFT 技术有 LoRA、Adapter Tuning 、Prefix Tuning、Prompt Tuning。

（4）提示词微调 （Prompt Tuning）

通过精心制作的提示词模板和对应的输出实现模型的微调，而不对基础模型的任何参数进行更新，只更新 embedding 参数。

（5）RLHF （Reinforcement Learning from Human Feedback）

使用强化学习的方式直接优化带有人类反馈的语言模型，实现与人类价值观的对齐。

用 alpaca_cleaned 数据集进行微调。该数据集每一条均由 3 个字段组成，具体如下：Instruction：指令，指导模型进行下一步的操作或生成相应的文本。

Input：输入，通常是与指令相关联的输入文本或数据，用于在模型处理指令时进行参考或分析。输入可以为空，在空的情况下直接使用指令文本作为提示词。

Response：输出，通常是以指令和输入作为提示词生成的输出文本。

多模态大语言模型：Multi-Modal Large Language Model, MLLM

能够感知不同模态的输入，如图片、文字、声音等，并根据人类给出的指令，以自回归的方式学习上下文并生成回答。MLLM 使用了自然语言处理、计算机视觉和语音识别等技术，并将它们整合到一个系统中，从而能够更加准确地理解人类的语言和情感

模态：任意形式的信息都可以视为一个模态，如文本、图像、语音、光波、气味等

模型发展示例：模型绘画

单模态：AARON（通过机械臂进行作画） → DL（deep learning 深度学习）生成猫脸 → GAN（生成式对抗网络，生成器与判别器对抗） → DeepDream

多模态：DALL-E（文字→图片）、Midjourney Stable Diffusion

DALL-E1：GPT-3（Transformer） + VAE （自分编码器）；

DALL-E2：CLIP （视觉语言预训练模型） + Diffusion （扩散模型）；

DALL-E3：CLIP + VAE + Diffusion （扩散模型）；

扩散模型：扩散模型通过逆转扩散过程来生成新数据，也就是说，它通过在数据

中添加随机噪声，然后再逆转这个过程，从而从噪声数据中恢复原始数据分布，这样就能创造出新的数据

训练：1、前向过程（扩散过程）原始图像 G0 经过不断的加入高斯噪声生成模糊图像（即打马赛克），经过 n 步，最终生成一副不再扩散的稳定图像 Gn。

2、 反向过程（去噪过程）从图像 Mn=Gn 出发，通过带参数的模型 U-Net 一步步实现去噪过程恢复图像的原始状态，经过 n 步，最终获得去噪后的图像 M0。这个模型 U-Net 就是扩散模型

模型优化：通过训练模型 U-Net 的参数，满足前向噪声和反向预测噪声的分布残差 E 最小化，就获得了所需建立的扩散模型

图像特征：扩散模型假设最终那副稳定的图像就是该图的特征值。AI 将这个特征图与文字的词向量关联，AI 绘画的生成过程就是从文字的词向量映射到图像的特征向量，再从特征向量出发进行反复去噪，恢复出原始的图像，由于去噪的过程带有随机性，因此能生成不同的图像，但高级特征不会变，人还是人，只是外貌发生了改变